

Lecture 10

fprintf and fscanf functions

- fprintf writes output in the specified file.
- fprintf returns number of characters written in the file
- If there is error to write in file then fprintf return -1

Format

```
fprintf(fp, "control string", list);
```

Example -> `fprintf(fp, "%s %d %f", name, age, 7.5);`

continue...

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;
```

```
    char name[] = "Alice";
```

```
    int age = 22;
```

```
    fp = fopen("data.txt", "w");
```

```
    fprintf(fp, "%s %d %f", name, age, 7.5);
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

fscanf

- It reads formatted data from the file
- It return count of the successfully read items from the file.
- When end of the file reached it returns EOF

Syntax: `fscanf(fp, "control string", list);`

File name: data.txt

Alice 22 7.5

continue...

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;
```

```
    char name[20];
```

```
    int age;
```

```
    float marks;
```

```
    fp = fopen("data.txt", "r");
```

```
    fscanf(fp, "%s %d %f", name, &age, &marks);
```

```
    printf("%s %d %.1f", name, age, marks);
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

continue...

stdin → FILE *

stdout → FILE *

Error Handling During I/O Operations

1. Trying to read beyond the end-of-file mark
2. Device Overflow
3. Trying to use a file that has not been opened
4. Trying to perform operation on a file, when the file opened for another type of operation
5. Opening a file with an invalid filename
6. Attempting to write to a write-protected file

continue...

- `feof()` takes one argument as pointer to a file
- `feof()` it is a function which returns non-zero value if all the data of file has been read otherwise returns zero.

Ex -> `if(feof(fp))`

`printf("End of data");`

- `ferror()` it takes file pointer as its argument and returns a non-zero value integer if an error has been detected up to that point otherwise it return zero.

Ex -> `if(ferror(fp) != 0)`

`printf("An error has occurred.");`

continue...

- When a file is opened using `fopen()` function, a file pointer is returned. If the file cannot be opened for some reason, then the function returns a NULL pointer. This facility can be used to test whether a file has been opened or not.

Ex -> `if(fp == NULL)`

`printf("file could not be opened");`

- If file does not exist and `fopen()` is used to open that file in read mode then `fopen()` return null pointer.
- If file does not exist and `fopen()` is used to open that file in write mode then `fopen()` will create a file with specified name and returns pointer to that file

Random Access to Files

- We have discussed file functions that are useful for reading and writing data sequentially. However, when we want to access only a particular part of a file and not in reading other parts.
- This can be achieved with the help of the functions `fseek`, `ftell` and `rewind`
- `ftell` takes a file pointer and returns a number type long, that corresponds to the current position.

```
Ex -> n = ftell(fp);
```

- `rewind` takes a file pointer and resets the position to the start of the file.

```
Ex -> rewind(fp);
```

```
    n = ftell(fp);
```

continue...

- First byte in is numbered as 0, second byte as 1 and so on.
- rewind helps us reading a file more than once without having to close and open the file.
- **fseek** function is used to move the file position to a desired location within the file.

format : `fseek(file_pointer, offset, position);`

offset : specifies the number of bytes to be moved from the location specified by the position.

continue...

- The position can take one of the following three values:

value	position
0	Beginning of file
1	Current position
2	End of file

continue...

Operations of fseek Function

Statement	Meaning
<code>fseek(fp, 0L, 0)</code>	Go to the beginning
<code>fseek((fp, 0L, 1)</code>	Stay at the current position
<code>fseek((fp, 0L, 2)</code>	Go to the end of file, past the last character of the file
<code>fseek((fp, m, 0)</code>	Move to (m+1)th byte in the file
<code>fseek((fp, m, 1)</code>	Go forward by m bytes
<code>fseek((fp, -m, 1)</code>	Go backward by m bytes from the current position
<code>fseek((fp, -m, 2)</code>	Go backward by m bytes from the end. Position the file to the mth character from the end